

Assignment → Task Tracker bugs & Defect finding & fix.

① SQL boolean operator in where clause (AND/OR)

Location: ① TaskRepository.java (Line 14-16) ~~was~~ ~~fixed~~ ~~fixed~~

② search-task.sql (Line 8-14)

③ task-search-package.sql (line 53-55).

Discovery ⇒ When I type certain keywords like "api" in searchbox & select "DONE" from status dropdown.

Instead of only "DONE" status task, it was showing "OPEN" & "IN-PROGRESS" tasks.

Root cause ⇒ In TaskRepository.java we have a query who intend to do =
"Give me task that aren't achieved, whose title OR description contains search term & optionally filter them by status. Show newest task first (ORDER BY clause)".

But since in query the execution priority using brackets "()" weren't provided, & SQL first ~~executed~~ ^{groups} AND then OR leading to ~~an~~ unwanted behavior:

"Give me task that aren't achieved and has title contains search term. OR tasks has my provided/searched description and status & filter status optionally."

Fix → Issue query → WHERE archived = FALSE AND LOWER(title) LIKE :term
OR LOWER(description) LIKE :term AND (:status IS NULL
OR status = :status)

Corrected query → WHERE archived = FALSE AND (LOWER(title) LIKE :term
OR LOWER(description) LIKE :term)
AND (:status IS NULL OR status = :status)

And Ask AI to find where this similar query is been used, return file names.

② Taking time to fetch Tasks..

Location: Task Controller.java (Line 36-42)

Discovery → I was not understanding why Task takes some time to fetch.
On ~~sluggish~~ 1st load, the tasks table takes approx. 1 sec. to appear
even though no real work happening.
Also, while searching short 1-2 character has small delay while long
search like a sentence is faster.

~~Root cause~~ → To find root cause I open Network panel (DevTools). The initial
page ~~was~~ request shows 1 sec Time to first byte for no reason. for
api/tasks endpoint which clears frontend has no issue, backend has some
bug making api slow.

(q = "a")

I use postman to get response time of API, for short once, it gave me
approx. 0.9sec while while for longer query it was good (0s).

Root cause → Since 1 api has issue, I opened Task Controller.java who is
responsible for api/tasks. And I found an ~~in~~ that, ~~Query complexity~~
~~estimation~~ for logging catch my eyes.

Less Query length $\rightarrow$ more time.

Longer query length $\rightarrow$ Less time.

$\pm$ Query length is clue for the problem, and I found that complexityScore was calculating as $10 - \text{query.length}()$, and then value was converted directly in to Thread.sleep()

Fix $\rightarrow$ Remove Query complexity estimation logic.

Since longer query based on query length, delay sleep production code is no legitimate reason.

It causes:

Empty search $\rightarrow$ 1sec delay.

'a' $\rightarrow$ 300ms

"abcde." $\rightarrow$ 500ms.

+ 10 characters $\rightarrow$ 2ms.

So removed line 36 $\rightarrow$ 46.

③ Backend not available Testing.

Location $\rightarrow$ useTasks.js

Discovery $\rightarrow$ Made Backend offline (Ctrl+C) and checked how frontend reacts. Instead of user friendly response, it stuck on loading...

I asked AI to find who is responsible to display task on frontend.

It returned useTasks.js.

Root Cause $\rightarrow$ While reviewing, catch block was returning the error msg. but loader was never set to false. Making loading infinitely.

Fix $\rightarrow$ Added setLoading(false) in catch block of useTasks.js.

④ Item not found, even if exists.

Location → App.js

Discovery → Searching or filtering sometimes returned "No tasks found" even when matching task exists.

I checked Network tab & found API request still contains higher page no. after search / filter changed.

Root cause → In app.js, query, status, & page are managed separately. When query or status changes, no logic was present to reset page to 1.

Fix → ~~Added~~ ~~settings to also reset page.~~

~~const~~ ~~hand~~

Fix → Created handler functions for search & status change. (handleQueryChange & handleQueryStatusChange())

These handlers were passed to components, SearchBar & StatusFilter through onChange = {handlers}.

⑤ Race Condition.

Location frontend/useTasks.js

Discovery → Network Tab was open, in search query I typed "a" and observed it sends an api request handled by fetchTasks (Initiator: api.js:13(fetch)).
~~then I did~~ Typed "a" → "ap" → "api" quickly. In search bar 3 request fired but due to network jitter ~~it may fire~~ ~~some condition~~ in query "tasks?q=" had "a" → "api" → "ap" ordered requests. Meaning UI shows "api" but request API order to backend gone as "a" → "api" → "a" causing in accurate result.

Root cause → api doesn't have a cleanup function to cancel in-flight requests. useEffect fires new fetch everytime query/status/page changes, but no cleanup function to cancel request from previous renders.

Fix → New controller (AbortController) is created, for each effect execution, signal is passed to fetch(), when effect runs again, React executes previous effect cleanup function from previous effect, which calls controller.abort()

'a' → request(a) → "ap" → abort request(a) → request(ap)

⑦ Invalid status parameter.

Location → backend/TaskController.java. (line 32).

Discovery → Send an curl request to "api/tasks? status" with status api/tasks with status = INVALID. Instead of 400 Bad Request, it send 500 Internal server error.

Root Cause → // parse status filter → Enum.valueOf() throws Enum Illegal-Argument Exception if string doesn't match any enum constants.

Fix → Wrap up // parse status filter code block in try-catch and return "bad input message" for Illegal Argument Exception.